

RELATÓRIO

Trabalho de PSI

CURSO TÉCNICO DE PROGRAMAÇÃO E GESTÃO DE SISTEMAS INFORMÁTICOS

Professor: Breno Sousa

Alunos: Andrey, Nikita Vyshchypan, David Odintsov e Guram

08/10/2026

O relatório encontra-se em condições para ser apresentado

_____

Ciclo de Formação 2024/2027

Ano Letivo 2026/2027

ÍNDICE

ÍNDICE 1
ÍNDICE DE FIGURAS 1
INTRODUÇÃO 1
CAPÍTULO I — Lógica Operacional e Regras do Sistema 2
Capítulo II – Estrutura do projeto 3
Capítulo III - Lógica Operacional e Regras do Sistema 7
Capítulo IV - TRATAMENTO DE EXCEÇÕES E SANITIZAÇÃO DE DADOS 10
Capítulo V - CONCLUSÃO 11

ÍNDICE DE FIGURAS

Figura 1 - classe quartos 3
Figura 2 - funçao novo quarto 3
Figura 3 - classe Pessoas(ABC) 4
Figura 4 - classe Hotel 5
Figura 5 - importações 5
Figura 6 - função para inicialização 6
Figura 7 - função para encontrar recepcionista 7
Figura 8 - função para registrar hóspede 8
Figura 9 - Menu 9

INTRODUÇÃO

O presente trabalho reporta o desenvolvimento de uma aplicação de gestão hoteleira em linguagem Python. A solução foi concebida para responder integralmente aos requisitos do 1.º Trabalho Avaliativo do Módulo 11, focando-se na aplicação dos conhecimentos adquiridos em aula.

No âmbito do desenvolvimento do sistema de gestão para o hotel, a equipa iniciou o projeto com uma fase crucial de análise e conceção da arquitetura de orientação a objetos. O objetivo principal desta etapa foi estruturar de forma sólida os fundamentos do sistema, garantindo a extensibilidade, a reutilização de código e a aplicação correta dos princípios de programação orientada a objetos (POO) exigidos, tais como herança simples, herança múltipla, polimorfismo e o uso de classes abstratas.

CAPÍTULO I — Lógica Operacional e Regras do Sistema

No âmbito do desenvolvimento do sistema de gestão para o hotel, a equipa iniciou o projeto com uma fase crucial de análise e conceção da arquitetura de orientação a objetos. O objetivo principal desta etapa foi estruturar de forma sólida os fundamentos do sistema, garantindo a extensibilidade, a reutilização de código e a aplicação correta dos princípios de programação orientada a objetos (POO) exigidos, tais como **classes e métodos abstratos** (que servem de contratos obrigatórios para as subclasses) e os métodos construtores **__init__** para a inicialização organizada dos atributos. A partir daí, aplicam-se conceitos como herança simples, herança múltipla e polimorfismo como já foi dito na introdução.

1. **Gestão de Quartos:** Permite criar QuartoSimples e QuartoLuxo. O sistema controla a ocupação (livre/ocupado) e impede diárias com valores negativos.
2. **Gestão de Funcionários:** Regista Gerente, Rececionista, TecnicoManutencao e TecnicoRecepcao.
    - **Regra Chave:** É obrigatório existir pelo menos um Rececionista (ou TecnicoRecepcao) ativo para poder registar hóspedes ou fazer *check-in*.
3. **Atendimento e Reservas:** Com um rececionista disponível, regista-se o Hospede e associa-se um quarto livre e os dias de estadia. O método calcular_conta() calcula o custo total (dias × preço diário).
4. **Manutenção e Herança Múltipla:** O TecnicoManutencao regista reparações, enquanto o TecnicoRecepcao combina ambas as funções (receção e manutenção).
5. **Checkout:** Ao finalizar a estadia, calcula o total a pagar, libertar o quarto e remove a associação do hóspede ao quarto.

Capítulo II – Estrutura do projeto

Para cumprir a exigência de modularização do programa, a aplicação foi dividida em quatro ficheiros distintos. No ficheiro **quartos.py** encontra-se a classe abstrata *Quarto* e as respectivas subclasses *QuartoSimples* e *QuartoLuxo*.

```python
from abc import ABC, abstractmethod

class Quarto(ABC):
    def __init__(self, numero, tipo, preco_diaria):
        self.numero = numero
        self.tipo = tipo
        self._preco_diaria = 0
        self.preco_diaria = preco_diaria
        self.ocupado = False
```

Figura 1 - classe quartos

```python
def novo_quarto(hotel):
    tipo = ler_num("1- Simples , 2- Luxo: ")
    num = ler_num("Número: ")
    preco = ler_num("Preço/diária: ", float)


    if hotel.find_quarto(num):
        print("Já existe um quarto com esse número."
        return
    try:
        if tipo == 1:
            quarto = QuartoSimples(num, preco)
        if tipo == 2:
            quarto = QuartoLuxo(num, preco)

        hotel.adicionar_quarto(quarto)
        print(f"Quarto {num} criado com sucesso.")
    except ValueError:
        print("Não foi possível criar o quarto.")
```

figura 2 - funçao novo quarto

Esta abordagem garante que não existem números de quarto duplicados e que os dados introduzidos pelo utilizador respeitam os critérios definidos na classe abstrata e respetivas subclasses.

Por sua vez, o ficheiro **pessoas.py** concentra a classe abstrata *Pessoa*, a subclasse *Hóspede* e a hierarquia completa de colaboradores (*Funcionário*, *Gerente*, *Recepcionista*, *TecnicoManutencao* e *TecnicoRecepcao*).

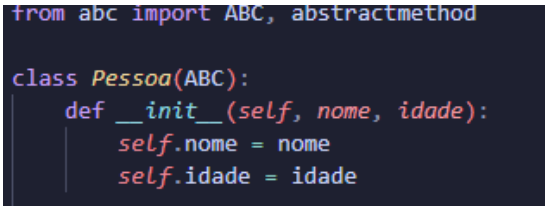


figura 3 - classe Pessoas(ABC)

Classe Pessoa (Abstrata): Define a base com os atributos protegidos/privados para nome e idade, utilizando propriedades (@property e @setter) para assegurar validações (como impedir idades negativas).Inclui o método abstrato mostrar_informacoes(self).

Classe Hospede: Herda de Pessoa e acrescenta atributos específicos como os dias_estadia e o quarto atribuído. Contém métodos para associar quartos, calcular o valor total da estadia e efetuar o *checkout*.

Hierarquia de Funcionários: A classe Funcionario estende Pessoa e dá origem a cargos específicos:

- **Gerente:** Acrescenta o cálculo de salário com bónus e a capacidade de gerar relatórios gerais dos funcionários.
- **Rececionista:** Inclui a gestão de turnos (manhã, tarde, noite) e métodos para registar e listar hóspedes.

- **TecnicoManutencao:** Focado na área técnica e especialidades de manutenção.
- **TecnicoRecepcao:** Um exemplo prático de **herança múltipla**, herdando simultaneamente de Rececionista e TecnicoManutencao para acumular funções administrativas e técnicas.

O ficheiro **hotel.py** define a classe gestora *Hotel*, que centraliza as listas de dados e os métodos de pesquisa necessários para o funcionamento do sistema.



Figura 4 - classe Hotel

Por fim, o ficheiro **main.py** é responsável pelo fluxo principal da aplicação, gerindo o menu em consola e a validação dos dados introduzidos pelo utilizador.



Figura 5 - importações

O ficheiro `main.py` gere o fluxo principal e o menu da aplicação. Como não possui classes próprias, utiliza as instruções `from ... import ...` para unir os vários módulos do projeto.

`from hotel import Hotel`: importa a classe de gestão global do hotel e **`from quartos import QuartoSimples, QuartoLuxo`**: importa os tipos de quartos disponíveis e etc…

Ponto de Entrada da Aplicação

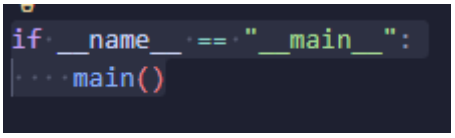


Figura 6 - função para inicialização

A inclusão desta estrutura assegura que o fluxo do menu só é iniciado quando o ficheiro é executado diretamente como script principal.

Esta é uma boa prática fundamental em Python, pois impede que o menu seja acionado acidentalmente caso o `main.py` venha a ser importado como módulo noutros scripts ou em testes unitários. Dessa forma, garante-se uma separação clara entre a definição da lógica e a execução efetiva da aplicação.

Capítulo III - Lógica Operacional e Regras do Sistema

Validação Prática no Código (main.py e hotel.py)

Para aplicar a regra de que só é possível registar hóspedes se existir um rececionista, o programa utiliza métodos de verificação direta nas classes e no menu principal:

- **Na classe `Hotel` (`hotel.py`):** Foi implementado o método `find_recepcionista(self)`, que percorre a lista de funcionários através de um ciclo e utiliza a função `isinstance()` para detetar se algum deles pertence à classe `Recepcionista` (ou subclasses compatíveis):

```
def find_recepcionista(self):
    for funcionario in self.funcionarios:
        if isinstance(funcionario, Recepcionista):
            return funcionario
    return None
```

Figura 7 - funçao para encontrar recepcionista

- **No menu principal (main.py):** A função registar_hospede(hotel) invoca este método antes de avançar para a recolha de dados do cliente. Caso nenhum rececionista seja encontrado (not recep), a operação é imediatamente cancelada com uma mensagem de erro, garantindo o cumprimento estrito da regra de negócio:

```python
def registar_hospede(hotel):
    recep = hotel.find_recepcionista()
    if not recep:
        print("Erro: Registe primeiro um recepcionista.")
        return
    nome = input("Nome: ")
    idade = ler_num("Idade: ")
    dias = ler_num("Dias de estadia: ")
    try:
        hospede = Hospede(nome, idade, dias)
        print(recep.registrar_hospede(hospede, hotel.hospedes))
    except ValueError:
        print("Não foi possível registar o hóspede.")
```

Figura 8 - funçao para registrar hospede

O Ciclo Principal da Aplicação (main())

O coração interativo do programa reside na função main(), que instancia a classe central Hotel e implementa um ciclo infinito de repetição (while True) conjugado com uma estrutura de seleções múltiplas (if-elif-else).

Através deste menu em consola, o utilizador pode navegar pelas várias operações do sistema de forma contínua até escolher sair (opcao == "0"):

```python
def main():
    hotel = Hotel("Hotel Central")

    while True:
        menu()
        opcao = input("Opção: ").strip()
        if opcao == "1":
            registar_hospede(hotel)
        elif opcao == "2":
            registar_funcionario(hotel)
        elif opcao == "3":
            reservar_quarto(hotel)
        elif opcao == "4":
            for quarto in hotel.quartos:
                print(quarto.mostrar_informacoes())
        elif opcao == "5":
            recep = hotel.find_recepcionista()
            if recep:
                recep.listar_hospedes(hotel.hospedes)
            else:
                print("Registe primeiro um recepcionista.")
        elif opcao == "6":
            for f in hotel.funcionarios:
                print(f.mostrar_informacoes())
        elif opcao == "7":
            fazer_checkout(hotel)
        elif opcao == "8":
            registar_reparo(hotel)
        elif opcao == "9":
            novo_quarto(hotel)
        elif opcao == "0":
            print("a sair")
            break
```

Figura 9 - Menu

Capítulo IV - TRATAMENTO DE EXCEÇÕES E SANITIZAÇÃO DE DADOS

Para garantir que a aplicação não vá abaixo por erros de digitação durante os testes no VS Code, a camada de entrada foi protegida com a função auxiliar

```
def ler_num(texto, tipo=int):
    while True:
        try:
            return tipo(input(texto).replace(",", "."))
        except ValueError:
            print("Número inválido.")
```

Mecanismos de Proteção Implementados:

Para garantir a estabilidade do programa e prevenir falhas durante a navegação, foram implementados diversos mecanismos de validação no `main.py`. O tratamento da exceção `ValueError` impede que a aplicação termine inesperadamente quando são introduzidas letras em campos que exigem números inteiros ou decimais, mantendo o utilizador num ciclo até que introduza um valor válido. Adicionalmente, de modo a suportar a convenção nacional de numeração, o método `.replace(",", ".")` converte automaticamente entradas com vírgula (como "120,50") para a sintaxe de decimais aceite pelo Python ("120.50"). Por fim, a aplicação do método `.strip()` nas escolhas do menu elimina quaisquer espaços em branco inseridos por engano antes ou depois da opção selecionada.

Capítulo V - CONCLUSÃO

A implementação do Sistema de Gestão Hoteleira cumpriu rigorosamente todos os requisitos pedagógicos do Módulo 11. A divisão do programa em quatro módulos organizados (pessoas.py, quartos.py, hotel.py e main.py) permitiu separar as regras de negócio da camada de interatividade.

A combinação de validações com decoradores @property/@setter, tratamento de exceções com try-except e suporte a herança simples e múltipla resultou numa aplicação backend resiliente, clara e expansível.